Problem 1

a) $f(x) = |x|^{\frac{3}{2}} = \begin{cases} x^{\frac{3}{2}}, & x \geq 0 \\ (-x)^{\frac{3}{2}}, & x < 0 \end{cases}$

$\therefore f'(x) = \begin{cases} \frac{3}{2} x^{\frac{1}{2}}, & x \geq 0 \\ -\frac{3}{2}(-x)^{\frac{1}{2}}, & x < 0 \end{cases}$

Assuming that f is Lipschitz smooth

$\therefore$ For $x > 0$, $y = 0$

$\|\nabla f(x) - \nabla f(y)\| = \frac{3}{2} x^{\frac{1}{2}}$

$\because \|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|$ with constant $L > 0$, $\|x - y\| = x$

$\therefore Lx \geq \frac{3}{2} x^{\frac{1}{2}} \quad \therefore L \geq \frac{3}{2} x^{-\frac{1}{2}}$

$\because$ When $x \to 0$, $\frac{3}{2} x^{-\frac{1}{2}} \to \infty$

$\therefore$ Contradiction

$\therefore f$ is not Lipschitz smooth.

b) $\because x_1 = x_0 - \bar{\alpha} \nabla f(x_0)$,

$\nabla f(x_0) = \begin{cases} \frac{3}{2} x_0^{\frac{1}{2}}, & x_0 \geq 0 \\ -\frac{3}{2}(-x_0)^{\frac{1}{2}}, & x_0 < 0 \end{cases}$

For $x_0 \geq 0$:

$x_1 = x_0 - \bar{\alpha} \cdot \frac{3}{2} x_0^{\frac{1}{2}}$

$\therefore f(x_1) = \left| x_0 - \bar{\alpha} \frac{3}{2} x_0^{\frac{1}{2}} \right|^{\frac{3}{2}}$

$f(x_0) = x_0^{\frac{3}{2}}$

$\therefore$ For $\bar{\alpha} > \frac{4}{3} x_0^{\frac{1}{2}}$, $x_0 - \bar{\alpha} \frac{3}{2} x_0^{\frac{1}{2}} < 0$

$\therefore f(x_1) = (\bar{\alpha} \frac{3}{2} x_0^{\frac{1}{2}} - x_0)^{\frac{3}{2}} > (2x_0 - x_0)^{\frac{3}{2}} = x_0^{\frac{3}{2}} = f(x_0)$

$\therefore$ For any $\bar{\alpha}$, we can find $0 < x_0 < \frac{9}{16} \bar{\alpha}^2$ for which $f(x_1) \geq f(x_0)$

Problem 2.

a) $f(x) = x^2 + 3\sin^2 x$

$\because \sin^2 x = \frac{1 - \cos 2x}{2}$

$\therefore f(x) = x^2 - \frac{3}{2}\cos 2x + \frac{3}{2}$

$\therefore f'(x) = 2x + 3\sin 2x$

$f''(x) = 2 + 6\cos 2x$

$\therefore f''(\frac{\pi}{2}) = 2 - 6 \times 1 = -4 < 0$

$\therefore$ The function is not convex

$\because f(x) = x^2 + 3\sin^2 x$, $x^2 \geq 0$, $\sin^2 x \geq 0$, $f(x) \geq 0$

$\because$ Only when $x = 0$, $x = \sin^2 x = 0$, $f(x) = 0$.

$\therefore x = 0$ is a global minimizer.

(b) $\because f''(x) = 2 + 6\cos 2x$, $\cos 2x \in [-1, 1]$

$\therefore f''(x) \leq 2 + 6 = 8$

$\therefore \|\nabla^2 f(x)\| \leq 8$

$\therefore L = 8$, the function is Lipschitz smooth.

(c) $\because g(y) \leq g(x) + \nabla g(x)^T (y - x) + \frac{L}{2}\|y - x\|^2$

$\therefore$ For $x = x_k$, $y = x_{k+1}$:

$f(x_{k+1}) \leq f(x_k) + f'(x_k)(x_{k+1} - x_k) + \frac{L}{2}\|x_{k+1} - x_k\|^2$

$\because x_{k+1} = x_k - \bar{\alpha} f'(x_k) \Rightarrow x_{k+1} - x_k = -\bar{\alpha} f'(x_k)$

$\therefore \Rightarrow f(x_{k+1}) \leq f(x_k) - \bar{\alpha}|f'(x_k)|^2 + \frac{L}{2} \cdot \bar{\alpha}^2 |f'(x_k)|^2$

$\therefore f(x_{k+1}) \leq f(x_k) - (\bar{\alpha} - \frac{L}{2}\bar{\alpha}^2)|f'(x_k)|^2$

(d) $\because |f'(x)|^2 \geq \frac{1}{16}(f(x) - f(x^*)), \forall x$, $f(x_{k+1}) \leq f(x_k) - (\bar{\alpha} - \frac{L}{2}\bar{\alpha}^2)|f'(x_k)|^2$

$\therefore f(x_{k+1}) \leq f(x_k) - \frac{1}{16}(\bar{\alpha} - \frac{L}{2}\bar{\alpha}^2)(f(x_k) - f(x^*))$

$\therefore f(x_{k+1}) - f(x^*) \leq f(x_k) - f(x^*) - \frac{1}{16}(\bar{\alpha} - \frac{L}{2}\bar{\alpha}^2)(f(x_k) - f(x^*))$

$\therefore f(x_{k+1}) - f(x^*) \leq (f(x_k) - f(x^*))\left[1 - \frac{1}{16}(\bar{\alpha} - \frac{L}{2}\bar{\alpha}^2)\right]$

For $\bar{\alpha} = \frac{1}{L} \Rightarrow \bar{\alpha} - \frac{L}{2}\bar{\alpha}^2 = \frac{1}{L} - \frac{1}{2L} = \frac{1}{2L} = \frac{1}{16}$

$\therefore f(x_{k+1}) - f(x^*) \leq \frac{255}{256}(f(x_k) - f(x^*))$

$\eta = \frac{255}{256} \in (0, 1)$

3.

```python
import numpy as np

def f(x):
    return 4 + x**2 - x**4

def bisection_method(a, b, tol=1e-4):
    iterations = 0
    while (b - a) > tol:
        c = (a + b) / 2
        df_a = 2*a - 4*a**3
        df_c = 2*c - 4*c**3

        if df_a * df_c < 0:
            b = c
        else:
            a = c
        iterations += 1
    return (a + b) / 2, iterations

def golden_section_method(a, b, tol=1e-4):
    golden_ratio = (np.sqrt(5) - 1) / 2
    iterations = 0

    c = b - golden_ratio * (b - a)
    d = a + golden_ratio * (b - a)

    while abs(c - d) > tol:
        if f(c) < f(d):
            b = d
        else:
            a = c

        c = b - golden_ratio * (b - a)
        d = a + golden_ratio * (b - a)
        iterations += 1

    return (a + b) / 2, iterations

a, b = -1, 0.5

bisect_sol, bisect_iters = bisection_method(a, b)
golden_sol, golden_iters = golden_section_method(a, b)

print("Bisection Method:")
print(f"Approximate solution: {bisect_sol:.6f}")
print(f"Iterations required: {bisect_iters}")

print("\nGolden Section Method:")
print(f"Approximate solution: {golden_sol:.6f}")
print(f"Iterations required: {golden_iters}")


if bisect_iters < golden_iters:
    print("\nBisection method required fewer iterations.")
elif golden_iters < bisect_iters:
    print("\nGolden section method required fewer iterations.")
else:
    print("\nBoth methods required the same number of iterations.")
```

```
q3.py
Bisection Method:
Approximate solution: -0.000015
Iterations required: 14
Approximate solution: -0.000015
Iterations required: 14
Iterations required: 14

Golden Section Method:
Approximate solution: 0.000017
Iterations required: 17

Bisection method required fewer iterations.
```

4. a) True.

If x_k is not a stationary point

$\Rightarrow \nabla f(x_k) \neq 0$

$\therefore \forall i \ \nabla f(x_k) = \begin{cases} -\nabla f(x_k), & \nabla f(x_k) > 0 \\ \nabla f(x_k), & \nabla f(x_k) < 0 \end{cases}$

$\therefore \forall i \ \nabla f(x_k) < 0$

$\therefore v$ is a descent direction as long as x_k is not a stationary point.

(b) True.

With backtracking line search:

$f(x_{k+1}) - f(x_k) \leq \gamma \alpha_k \nabla f(x_k)^T d^k$

$\gamma > 0, \alpha_k > 0, \nabla f(x_k)^T d^k < 0$

$\therefore$ As long as $\nabla f(x^k) \neq 0$, $f(x_{k+1}) < f(x_k)$

$\therefore f(x_{k+1}) < f(x_k)$ as long as x_k is not a stationary point

(c) True.

By exact line search, we choose α with minimize $\phi(\alpha) = f(x_k - \alpha \nabla f(x_k))$

$\because \phi'(0) = -\|\nabla f(x_k)\|^2$

$\therefore$ As long as $\nabla f(x_k) \neq 0$, $\phi'(0) < 0$,

$\therefore \exists \alpha > 0$ such that $\phi(\alpha) < \phi(0)$

$\therefore f(x_{k+1}) = \min_\alpha f(x_k - \alpha \nabla f(x_k)) < f(x_k)$

$\therefore f(x_{k+1}) < f(x_k)$ as long as x_k is not a stationary point.

(d) False.

A counter example:

For $f(x) = x^4 - 2x^2 \ x \in \mathbb{R}$.

$f'(x) = 4x^3 - 4x \quad f'(x) = 12x^2 - 4$

$\therefore$ When $f'(x) = 0$, $x = 0$ or $x = \pm 1$

$f''(0) < 0$, $f''(1) = f''(-1) > 0$

$\therefore \pm 1$ are local minimizer.

But if we start from $x_0 = 0$

$f'(x_0) = f'(0) = 0$. x_0 is a stationary point, so the search won't be updated.

$\therefore$ Gradient descent won't converge to a local minimizer.

5. $\min_x f(x) = x^T A x$, $A = \begin{bmatrix} 2 & 0 \\ 0 & 5 \end{bmatrix}$

a) $\nabla f(x) = 2Ax = \begin{bmatrix} 4 & 0 \\ 0 & 10 \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 4x_1 \\ 10x_2 \end{bmatrix}$

$\therefore \nabla f(x_0) = \begin{bmatrix} 4 \\ 10 \end{bmatrix}$

① Using a constant step size of 0.1:

$\therefore x_1 = x_0 - 0.1 \cdot \nabla f(x_0) = \begin{bmatrix} 1 \\ 1 \end{bmatrix} - \begin{bmatrix} 0.4 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.6 \\ 0 \end{bmatrix}$

$\therefore \nabla f(x_1) = \begin{bmatrix} 2.4 \\ 0 \end{bmatrix}$

$\therefore x_2 = x_1 - 0.1 \nabla f(x_1) = \begin{bmatrix} 0.36 \\ 0 \end{bmatrix}$

$\therefore x_2 = \begin{bmatrix} 0.36 \\ 0 \end{bmatrix}$

② Using exact line search.

$\because \nabla f(x_0) = \begin{bmatrix} 4 \\ 10 \end{bmatrix}$

$\therefore g(\alpha) = f(x_0 - \alpha \nabla f(x_0)) = f\left(\begin{bmatrix} 1-4\alpha \\ 1-10\alpha \end{bmatrix}\right) = 2(1-4\alpha)^2 + 5(1-10\alpha)^2$

$\therefore g'(\alpha) = -16(1-4\alpha) - 100(1-10\alpha)$

$= 64\alpha - 16 + 1000\alpha - 100 = 1064\alpha - 116$

$\therefore$ When $g'(\alpha) = 0 \Rightarrow \alpha = \frac{116}{1064} = \frac{29}{266}$

$\because g''(\alpha) > 0$

$\therefore \alpha = \frac{29}{266} \approx 0.109$

$\therefore x_1 = x_0 - \alpha \nabla f(x_0) = \begin{bmatrix} 1 \\ 1 \end{bmatrix} - \begin{bmatrix} \frac{29}{266} \times 4 \\ \frac{29}{266} \times 10 \end{bmatrix} = \begin{bmatrix} 0.564 \\ -0.090 \end{bmatrix}$

③ Using backtracking line search:

$\because \gamma = \frac{1}{2}$, $\sigma = 0.2$, $\nabla f(x) = \begin{bmatrix} 4 \\ 10 \end{bmatrix}$, $\nabla f(x_0) = \begin{bmatrix} 4 \\ 10 \end{bmatrix}$.

We choose $\alpha = 1$ as starting α

$\therefore x_0 - \alpha \nabla f(x_0) = \begin{bmatrix} -3 \\ -9 \end{bmatrix}$, $f(x_0) = 7$, $\gamma \alpha \|\nabla f(x_0)\|^2 = \frac{1}{2} \cdot 116 = 58$

$\therefore f(x_0 - \alpha \nabla f(x_0)) = 2 \times 9 + 5 \times 81 = 423$

$\therefore f(x_0 - \alpha \nabla f(x_0)) > f(x_0) + \gamma \alpha \|\nabla f(x_0)\|^2$

$\therefore \alpha \to 60\alpha = 0.2$

$\therefore x_0 - \alpha \nabla f(x_0) = \begin{bmatrix} 0.2 \\ -1 \end{bmatrix}$, $\gamma \alpha \|\nabla f(x_0)\|^2 = 11.6$

$\therefore f(x_0 - \alpha \nabla f(x_0)) = 5.08 > 7 - 11.6$

$\therefore \alpha \to 6\alpha = 0.04$

$\therefore x_0 - \alpha \nabla f(x_0) = \begin{bmatrix} 0.84 \\ 0.6 \end{bmatrix}$, $\gamma \alpha \|\nabla f(x_0)\|^2 = 2.32$

$\therefore f(x_0 - \alpha \nabla f(x_0)) = 3.2112 < 7 - 2.32 = 4.68$

$\therefore x_1 = \begin{bmatrix} 0.84 \\ 0.6 \end{bmatrix}$.

```python
import numpy as np

def f(x, A):
    return x.T @ A @ x

def gradient(x, A):
    return 2 * A @ x

A = np.array([[2, 0], [0, 5]])
x0 = np.array([1.0, 1.0])
max_iter = 10


def constant_step(x0, A, alpha=0.1, max_iter=max_iter):
    x = x0.copy()
    for _ in range(max_iter):
        grad = gradient(x, A)
        x = x - alpha * grad
    return x, f(x, A)


def exact_line_search(x0, A, max_iter=max_iter):
    x = x0.copy()
    for _ in range(max_iter):
        grad = gradient(x, A)
        numerator = grad.T @ grad
        denominator = 2 * grad.T @ A @ grad
        alpha = numerator / denominator
        x = x - alpha * grad
    return x, f(x, A)
```

```python
def backtracking_line_search(x0, A, gamma=0.5, sigma=0.2, max_iter=max_iter):
    x = x0.copy()
    for _ in range(max_iter):
        grad = gradient(x, A)
        alpha = 1.0
        while True:
            x_new = x - alpha * grad
            lhs = f(x_new, A)
            rhs = f(x, A) - gamma * alpha * (grad.T @ grad)
            if lhs <= rhs:
                break
            alpha *= sigma
        x = x_new
    return x, f(x, A)


print("Constant step (α=0.1):")
x_const, f_const = constant_step(x0, A)
print(f"Final x: {x_const}, Final f(x): {f_const:.6f}\n")

print("Exact line search:")
x_exact, f_exact = exact_line_search(x0, A)
print(f"Final x: {x_exact}, Final f(x): {f_exact:.6f}\n")

print("Backtracking line search:")
x_back, f_back = backtracking_line_search(x0, A)
print(f"Final x: {x_back}, Final f(x): {f_back:.6f}")
```

```
Constant step (α=0.1):
Final x: [0.00604662 0.        ], Final f(x): 0.000073

Exact line search:
Final x: [8.44236534e-06 8.44236534e-06], Final f(x): 0.000000

Backtracking line search:
Final x: [ 0.04164315 -0.0100777 ], Final f(x): 0.003976
PS C:\Users\lenovo\Desktop\大二下\MAT3007\mat>
```